CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G4
System Integration and Architecture

System Design Document (SDD)

Project Title: Voxa

Prepared By: Cortes, Raven Fatima L.

Version: [e.g. 1.0]

Date: [Current Date]
Status: [Draft/Final]

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	02/2/2026	Cortes	Initial draft	Draft
0.2	02/5/2026	Cortes	Added Core User Journeys	Review
0.3	02/18/2026	Cortes	Continued until Non-Functional Req	Review
0.4				Revised
0.5				Final
1	[Date]	Cortes	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY..... 4

1.0 INTRODUCTION..... 5

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION..... 5

3.0 NON-FUNCTIONAL REQUIREMENTS..... 9

4.0 SYSTEM ARCHITECTURE..... 11

5.0 API CONTRACT & COMMUNICATION..... 12

6.0 DATABASE DESIGN..... 17

7.0 UI/UX DESIGN..... 18

8.0 PLAN..... 21

EXECUTIVE SUMMARY

1.1 Project Overview

Voxa is a simplified online feedback and survey system that allows students to submit feedback and respond to surveys through a centralized digital platform. The application enables users to register and log in, view available surveys on a dashboard, submit responses, and manage their profiles, providing faculty and administrators with an efficient way to collect and review student input.

1.2 Objectives

1. Develop a fully functional online feedback and survey system with secure user registration and authentication
2. Implement a centralized dashboard for viewing and accessing available surveys
3. Enable students to answer and submit feedback or survey responses efficiently
4. Allow users to view their submitted responses for transparency and record-keeping
5. Ensure reliable data handling to support faculty and administrators in collecting and reviewing feedback

1.3 Scope

Included Features:

- User registration and authentication (email and password)
- Role-based access for students and faculty/administrators
- User profile management
- Dashboard
- Survey creation and management
- Viewing of submitted responses and basic management of collected responses
- Basic survey question types (multiple choice and short answer)
- Responsive web-based user interface accessible on common devices
- Logout and session management

Excluded Features:

- Real-time chat or discussion threads

- Anonymous feedback submission
- File or media attachments in survey responses
- Advanced analytics, reporting, or data visualization.
- AI-based sentiment analysis or automated insights

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the Voxa – Voice + Expression system. It outlines the system requirements, core features, application scope, and overall design approach to guide the development and implementation of an online feedback and survey platform.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: Voxa
Domain: Academic Feedback System
Primary Users: Students and Faculty/Administrators

Problem Statement: Educational institutions need a simple and efficient way to collect student feedback and survey responses without relying on manual or fragmented methods.
Solution: A centralized online feedback and survey application that enables students to easily submit responses and allows faculty and administrators to efficiently collect, review, and manage feedback through a streamlined digital platform.

2.2 Core User Journeys

Journey 1: Student Submits Feedback / Survey (First-Time User)

1. User visits the Voxa web application
2. Clicks “Sign Up” and creates an account
3. Logs in using registered credentials
4. Views the dashboard displaying available surveys
5. Selects a survey or feedback form

6. Answers the survey questions
7. Submits the response successfully

Journey 2: Student Submits Feedback / Survey (Returning User)

1. User logs in with existing credentials
2. Accesses the dashboard
3. Selects a new or ongoing survey
4. Completes and submits the survey
5. Views previously submitted responses

Journey 3: Faculty / Administrator Manages Surveys and Responses

1. Faculty/Administrator logs in with authorized credentials
2. Navigates to the survey management section
3. Creates a new survey with basic question types
4. Publishes the survey to make it available to students
5. Views and reviews submitted responses
6. Logs out of the system

2.3 Feature List (MoSCoW)

MUST HAVE

1. User Registration & Login
2. Role-Based Access (Student/Faculty)
3. Dashboard View of Available Surveys
4. Take Survey (MCQ & Short Answer)
5. Submit Survey Response
6. View My Previous Responses
7. Create New Survey (Faculty)
8. Publish/Unpublish Survey (Faculty)
9. View Responses to a Survey (Faculty)
10. Secure Logout

SHOULD HAVE

1. Edit Profile Information
2. Basic Survey Analytics (e.g., response count)
3. Pagination for Long Survey Lists
4. Confirmation Email on Registration
5. "Not Now" option to postpone a survey

COULD HAVE

1. Password Reset Functionality
2. Survey Expiration Dates
3. Draft Survey Capability
4. UI Theme Selector

WON'T HAVE

1. Anonymous Feedback
2. File Attachments
3. Real-time Chat/Discussions
4. Advanced Data Visualization
5. AI-based Sentiment Analysis

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Login, Register, Forgot Password
- **Fields:** Email, Password, Confirm Password (Registration), Full Name (Registration), Role (Pre-selected/Assigned during registration/setup).
- **Validation:** Email format, password strength (min. 8 characters), password confirmation match, required fields.
- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout
- **Security:** Passwords hashed using BCrypt. JWT (JSON Web Tokens) for session management. HTTPS for all communication.

Feature: Survey Dashboard (Student)

- **Screens:** Main Dashboard
- **Display:** A list or grid of surveys available to the student. Each item should show the survey title, description (optional), and a status (e.g., "New", "In Progress", "Completed").
- **Search:** Ability to filter surveys by status (e.g., Available, Completed).
- **API Endpoints:** GET /api/v1/surveys?status=available

Feature: Take Survey

- **Screens:** Survey Taking Screen
- **Functions:** Display survey title and questions sequentially or on one page, Support for Multiple Choice (single select) and Short Answer text fields, Progress indicator (e.g., "Question 3 of 10"), Submit button to finalize responses.
- **Validation:** Ensure all required questions are answered before submission.
- **API Endpoints:** GET /api/v1/surveys/{surveyId} , POST /api/v1/surveys/{surveyId}/responses

Feature: View My Responses (Student)

- **Screens:** My Responses / History Screen
- **Functions:** View a list of surveys the student has already submitted.
- **Display:** Upon selecting a survey from the list, the student can see the questions and their provided answers in a read-only format.
- **API Endpoints:** GET /api/v1/user/responses

Feature: Survey Management (Faculty/Admin)

- **Screens:** Admin Survey Management Dashboard, Create/Edit Survey Screen.
- **Functions:** View a list of all created surveys (with status: Draft, Published), Create a new survey by providing a title and description, Add questions (Multiple Choice or Short Answer). For MCQs, add options, Edit or delete a survey (draft only), Publish a survey to make it visible to students, Unpublish a published survey.
- **Access Control:** Access restricted to users with "FACULTY" or "ADMIN" role.
- **API Endpoints:** GET /api/v1/admin/surveys, POST /api/v1/admin/surveys, POST /api/v1/admin/surveys, DELETE /api/v1/admin/surveys/{surveyId}, POST

/api/v1/admin/surveys/{surveyId}/publish, POST
/api/v1/admin/surveys/{surveyId}/unpublish

Feature: View Survey Responses (Faculty/Admin)

- **Screens:** Survey Responses Screen
- **Functions:** For a specific survey, view a list of all individual student responses. See which students have responded.
- **API Endpoints:** GET /api/v1/admin/surveys/{surveyId}/responses, GET /api/v1/admin/surveys/{surveyId}/responses/{userId}

2.5 Acceptance Criteria

AC-1: Successful User Registration

- Given a user is on the Registration page
- When they enter a valid email, a strong password, and their full name, and click "Register"
- Then a new account is created, they are automatically logged in, and are redirected to the Student Dashboard.

AC-2: Student Successfully Submits a Survey

- Given a logged-in student is viewing an available survey
- When they answer all required questions and click "Submit"
- Then the responses are saved, a success message is displayed, and they are redirected to the dashboard where the survey is now marked as "Completed".

AC-3: Faculty Creates and Publishes a Survey

- Given a logged-in faculty member is on the Survey Management page
- When they click "Create New Survey", fill in the title, add at least one question, and click "Publish"
- Then the survey is saved with a "Published" status and becomes immediately visible on all students' dashboards.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests
- API response time: ≤ 2 seconds for 95% of requests
- Dashboard load time: ≤ 2 seconds for authenticated users

- Support 100 concurrent users during peak periods
- Survey submission processing: ≤ 3 seconds
- Database queries complete within 500ms
- System uptime: 99.5% availability during academic semesters

3.2 Security Requirements

- HTTPS for all communications
- JWT token authentication with 1-hour expiration
- Password hashing with bcrypt (salt rounds = 12)
- SQL injection prevention through parameterized queries
- XSS protection via input sanitization and output encoding
- Rate limiting: 60 requests/minute per IP for public endpoints
- Admin endpoints require role verification (Faculty/Admin only)
- Session timeout after 30 minutes of inactivity
- Password minimum length: 8 characters with complexity requirements

3.3 Compatibility Requirements

- **Web Browsers:** Edge
- **Android:** Kotlin
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+, iOS 14+, Android 10+

3.4 Usability Requirements

- Complete survey submission within 3 minutes for new users
- Faculty can create and publish a new survey within 5 minutes
- WCAG 2.1 Level A compliance minimum (AA where feasible)
- Consistent navigation across all pages
- Clear error messages with recovery options
- Touch targets minimum 44x44px on mobile devices

- Keyboard navigation support for all interactive elements
- Form validation provides real-time feedback
- Survey progress indicator visible during completion
- Maximum 3 clicks to reach any core feature from dashboard

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram

```
┌──────────────────────┐   ┌──────────────────────┐
│  WEB CLIENT    │   │  MOBILE CLIENT  │
│   (React)      │   │  (Android)      │
│  • TypeScript  │   │  • Kotlin       │
│  • Tailwind CSS│   │  • Compose      │
└────────┬───────────┘   └────────┬───────────┘
         │                 │
         └────────┬────────┘
                  │
          ┌────────▼────────┐
          │  BACKEND API      │
          │  (Spring Boot)    │
          │  • Java 17        │
```

```
│  • Spring Security │
│  • JPA/Hibernate   │
└──────────┬──────────┘
           │
           ▼
┌──────────────────────┐
│ DATABASE             │
│ (PostgreSQL)         │
│ • Users table        │
│ • Products table     │
│ • Orders table       │
│ • Cart items table   │
└──────────────────────┘
```

Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, Jetpack Compose, Retrofit, Room
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

- **Base URL:** https://api.marketplace.com/api/v1
- **Format:** JSON for all requests/responses
- **Authentication:** Bearer token (JWT) in Authorization header
- **Response Structure:**

json

```
{
  "success": boolean,
  "data": object|null,
  "error": {
    "code": string,
    "message": string,
    "details": object|null
  },
  "timestamp": string
}
```

5.2 Endpoint Specifications

Authentication Endpoints

POST /auth/register

Body: {email, password, confirmPassword, fullName?}

Response: {user: {id, email, name}, token, refreshToken}

POST /auth/login

Body: {email, password}

Response: {user: {id, email, name, role}, token, refreshToken}

POST /auth/logout

Headers: Authorization: Bearer {token}

Response: {message: "Logged out successfully"}

Product Endpoints

GET /products

Query: ?page=1&limit=20&search=keyword&category=electronics

Response: {products: [...], pagination: {page, limit, total, pages}}

GET /products/{id}

Response: {product: {id, name, description, price, stock, imageUrl, category}}

POST /products (Admin only)

Body: {name, description, price, stock, imageUrl, category}

Response: {product: {...}}

PUT /products/{id} (Admin only)

Body: {name?, description?, price?, stock?, imageUrl?, category?}

Response: {product: {...}}

Cart Endpoints

GET /cart (Authenticated)

Response: {cart: {id, items: [...], total, itemCount}}

POST /cart/items

Body: {productId, quantity}

Response: {message: "Added to cart", cartItem: {...}}

PUT /cart/items/{itemId}

Body: {quantity}

Response: {message: "Cart updated", cartItem: {...}}

DELETE /cart/items/{itemId}

Response: {message: "Removed from cart"}

Order Endpoints

POST /orders

Body: {shippingAddress: {fullName, address, city, zipCode, country}}

Response: {order: {id, orderNumber, total, status, items: [...], createdAt}}

GET /orders

Response: {orders: [...]}

GET /orders/{id}

Response: {order: {...}}

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

```
json
{
  "success": false,
  "data": null,
```

```
  "error": {
    "code": "AUTH-001",
    "message": "Invalid credentials",
    "details": "Email or password is incorrect"
  },
  "timestamp": "2024-01-28T10:30:00Z"
}
```

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "VALID-001",
    "message": "Validation failed",
    "details": {
      "email": "Email is required",
      "password": "Must be at least 8 characters"
    }
  },
  "timestamp": "2024-01-28T10:30:00Z"
}
```


Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found

- DB-002: Duplicate entry
- BUSINESS-001: Insufficient stock
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD

```
USERS (1) ─────── (1) CARTS (1) ─────── (*) CART_ITEMS (*) ─────── (1) PRODUCTS
   │                    │
   │                    │
  (*)                  (*)
   │                    │
   │                    │
ORDERS (1) ─────── (*) ORDER_ITEMS (*) ──────┘
```

Detailed Relationships:

- **One-to-One:** User ↔ Cart (Each user has exactly one cart)
- **One-to-Many:** User → Orders (User can have multiple orders)
- **One-to-Many:** Cart → CartItems (Cart contains multiple items)
- **One-to-Many:** Order → OrderItems (Order contains multiple items)
- **Many-to-One:** CartItems → Product (Items reference products)
- **Many-to-One:** OrderItems → Product (Items reference products)

Key Tables:

1. **users** - User accounts and authentication
2. **products** - Product catalog information
3. **carts** - Shopping cart per user
4. **cart_items** - Items in shopping cart
5. **orders** - Customer orders

6. **order_items** - Items in each order
7. **refresh_tokens** - JWT refresh tokens

Table Structure Summary:

- **users:** id, email, password_hash, full_name, role, created_at
- **products:** id, name, description, price, stock, image_url, category
- **carts:** id, user_id, created_at
- **cart_items:** id, cart_id, product_id, quantity
- **orders:** id, order_number, user_id, total, status, shipping_address
- **order_items:** id, order_id, product_id, product_name, quantity, price

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Note: This should be wireframes from Figma

Homepage (Product Listing)

Header: [Logo] [Search Bar] [Cart Icon] [User Menu]

Content: Product Grid (3 columns desktop)

Each Product Card: Image, Name, Price, "Add to Cart" button

Footer: Links, Copyright

Product Detail Page

Back Button

Product Image (large)

Product Name and Price

Description

Quantity Selector (1-10)

"Add to Cart" and "Buy Now" buttons

Product Specifications

Shopping Cart Page

Cart Title

List of Cart Items (Image, Name, Quantity, Price, Remove)

Order Summary: Subtotal, Shipping, Tax, Total

"Continue Shopping" and "Proceed to Checkout" buttons

Checkout Page

Shipping Address Form

Order Review (Items, Prices, Totals)

"Place Order" button

Terms and Conditions note

Admin Dashboard

Sidebar Navigation: Dashboard, Products, Orders, Users

Product Management: Add New button, Product list with Edit/Delete

Order Management: Order list with status filters

7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma

Bottom Navigation

[🏠 Home] [🔍 Search] [🛒 Cart] [👤 Profile]

Home Screen

Search Bar

Product Grid (2 columns)

Swipe gestures for quick actions

Pull to refresh

Product Detail Screen

Back arrow

Product image (swipeable gallery)

Product info

Quantity selector

"Add to Cart" fixed bottom button

Cart Screen

Edit mode for quantity updates

Swipe to remove items

Order summary sticky bottom

Checkout button

Checkout Flow

Step indicator: Cart → Shipping → Payment → Confirm

Address form (auto-complete)

Order summary

Place order button

Mobile-Specific Features:

- Touch-optimized buttons (min 44x44px)
- Gesture support (swipe, pull-to-refresh)
- Offline caching for product images
- Bottom navigation for main actions
- Simplified forms for mobile input

Design System:

- **Colors:** Primary (#2563EB), Secondary (#7C3AED), Success (#10B981), Error (#EF4444)
- **Typography:** Inter font family, responsive sizing
- **Spacing:** 8px grid system
- **Components:** Consistent buttons, inputs, cards, modals
- **Responsive:** Mobile-first approach, breakpoints at 640px, 768px, 1024px

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design

Day 5-6: UI/UX wireframes

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with dependencies

Day 2: Database configuration and entities

Day 3: JWT authentication implementation

Day 4: User management endpoints

Day 5: Product CRUD operations

Week 4: Core Features

Day 1: Cart functionality

Day 2: Order management

Day 3: Search and filtering

Day 4: Error handling and validation

Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup with TypeScript

Day 2: Authentication pages (login, register)

Day 3: Product listing page

Day 4: Product detail page

Day 5: Shopping cart implementation

Week 6: Complete Web Features

Day 1: Checkout flow

Day 2: Order history and confirmation

Day 3: Admin dashboard

Day 4: Responsive design polish

Day 5: API integration and testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Authentication screens

Day 3: Product browsing

Day 4: Shopping cart

Day 5: API service layer

Week 8: Complete Mobile App

Day 1: Checkout flow

Day 2: Order management

Day 3: UI polish and animations

Day 4: Testing on emulator/device

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across platforms

Day 2: Bug fixes and optimization

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (Railway/Heroku)

Day 2: Web app deployment (Vercel/Netlify)

Day 3: Mobile APK distribution

Day 4: Final testing

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Product catalog API (Week 3-4)
3. Shopping cart functionality (Week 4)
4. Checkout process (Week 6)
5. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements